CYLIXLAB · ASSESSMENT

Module 03

Natural Language Processing & Transformer Models

MCQ Assessment | 30 technical questions with green-check marked answers and full explanations

Assessment overview

Part	Questions	Topic
A	Q1–Q5	Tokenization & embeddings: subword units, embedding parameters, Word2Vec, cosine similarity, contextual vectors
B	Q6–Q10	Attention: scaling by $\sqrt{d_k}$, attention weights, $O(n^2)$ cost, multi-head dimensions, causal masking
C	Q11–Q15	Transformer blocks: positional encoding, FFN parameters, layer norm, cross-attention, sinusoidal values
D	Q16–Q20	Pre-trained models: BERT MLM and masking rule, GPT decoder-only, [CLS] token, T5 text-to-text
E	Q21–Q25	Fine-tuning & decoding: LoRA parameters, teacher forcing, beam search, temperature, top-p sampling
F	Q26–Q30	Evaluation & efficiency: perplexity, BLEU, F1 for NER, KV cache, attention memory scaling

HOW TO USE THIS ASSESSMENT

1. Cover the answers and choose one option per question (about 90 seconds each).
2. Check your answer; the correct option is highlighted and marked with a green check.
3. Read **every** explanation, including for questions you got right: the “why the others fail” notes target common misconceptions.
4. Score yourself with the answer key on the last page and revisit the matching theory section.

A Tokenization & Word Embeddings

Q1 TOKENIZATION

A model uses **subword tokenization** such as Byte-Pair Encoding (BPE). What is the main advantage over a pure word-level vocabulary?

- **A.** It guarantees that every word becomes exactly one token
- **B. Rare or unseen words are split into known subword pieces, so there are no out-of-vocabulary tokens while the vocabulary stays manageable**
- **C.** It removes the need for an embedding layer
- **D.** It makes the sequence length independent of the input text

Correct answer: B

Why it is correct. BPE starts from characters and repeatedly merges the most frequent adjacent pair. Frequent words stay whole, while a rare word like “tokenizational” becomes pieces such as “token”, “ization”, “al”. Any string can be represented, and the vocabulary is fixed (typically 30k–100k).

Why the others fail.

✗ **A.** Rare words are deliberately split into several tokens; only frequent words map to one token.
✗ **C.** Each subword ID still needs a learned embedding vector.
✗ **D.** Sequence length still grows with the text; splitting rare words can even make it longer.

Q2 EMBEDDINGS

An embedding layer maps a vocabulary of **30,000** tokens to vectors of dimension **512**. How many trainable parameters does it have?

- **A.** 15,360,512
- **B.** 30,512
- **C. 15,360,000**
- **D.** 7,680,000

Correct answer: C

Why it is correct. An embedding layer is a lookup table with one row per token: 30,000 × 512 = **15,360,000** weights. It has no bias, because a lookup simply selects a row; nothing is added afterwards.

Why the others fail.

✗ **A.** Adds 512 bias terms that an embedding lookup does not have.
✗ **B.** Adds the two sizes instead of multiplying them.
✗ **D.** Is exactly half the correct count, e.g. using 256 dimensions by mistake.

Q3 WORD2VEC

In Word2Vec, what does the **skip-gram** model learn to do during training?

- **A.** Predict the centre word from the average of its context words
- **B.** Classify whole sentences into topics
- **C.** Reconstruct the input sentence through a bottleneck
- ✓ **D. Predict the surrounding context words from the centre word**

Correct answer: D

Why it is correct. Skip-gram takes one centre word and is trained to predict the words within a window around it. Words that appear in similar contexts are pushed toward similar vectors, which is why the learned embeddings capture meaning.

Why the others fail.

✗ **A.** That is CBOW (continuous bag-of-words), the other Word2Vec variant.
✗ **B.** Word2Vec is trained on word co-occurrence, not on sentence labels.
✗ **C.** That describes an autoencoder, not Word2Vec.

Q4 EMBEDDINGS

Two word vectors are **a = [1, 2, 2]** and **b = [2, 1, 2]**. What is their cosine similarity?

- ✓ **A. ≈ 0.889**
- **B.** 8
- **C.** 0.5
- **D.** 1.0

Correct answer: A

Why it is correct. cos(a, b) = (a·b) / (‖a‖‖b‖). Dot product = 2 + 2 + 4 = 8. Both norms are √(1 + 4 + 4) = 3. So cos = 8 / 9 ≈ **0.889**: the vectors point in very similar directions.

Why the others fail.

✗ **B.** 8 is the raw dot product; it has not been divided by the norms.
✗ **C.** 0.5 corresponds to an angle of 60°, far larger than the real angle here (≈ 27°).
✗ **D.** 1.0 needs the vectors to point in exactly the same direction, which they do not.

Q5 EMBEDDINGS

Static embeddings such as Word2Vec or GloVe give the word **“bank”** the same vector in “river bank” and “bank loan”. Which approach fixes this?

- ✓ **A. Contextual embeddings from a model such as BERT, where each token’s vector depends on the whole sentence**
- ○ **B.** Increasing the Word2Vec embedding dimension
- ○ **C.** Using one-hot vectors instead of dense embeddings
- ○ **D.** Lower-casing all text before training

Correct answer: A

Why it is correct. In a Transformer encoder, every token’s output vector is computed by attending to the other tokens in the sentence, so “bank” next to “river” and “bank” next to “loan” get different representations. This handles polysemy, which static tables cannot.

Why the others fail.

✗ **B.** A bigger table is still one vector per word type, regardless of context.
✗ **C.** One-hot vectors are also context-free and carry no similarity information at all.
✗ **D.** Lower-casing merges spellings; it cannot separate two meanings of the same word.

B Attention Mechanism

Q6 ATTENTION

Scaled dot-product attention computes $\text{softmax}(QK^\top / \sqrt{d_k})\,V$. Why are the scores divided by $\sqrt{d_k}$?

- ○ **A.** To make the attention weights sum to 1
- ○ **B.** To reduce the number of parameters
- ○ **C.** To make the output dimension equal to d_k
- ✓ **D. Dot products grow in variance with d_k; without scaling, softmax saturates and its gradients become tiny**

Correct answer: D

Why it is correct. If the components of q and k have unit variance, q·k has variance d_k. For $d_k = 64$ the scores would be about 8× too spread out, pushing softmax toward one-hot outputs where gradients vanish. Dividing by $\sqrt{d_k}$ brings the variance back to about 1.

Why the others fail.

✗ **A.** Softmax already makes the weights sum to 1, with or without scaling.
✗ **B.** Scaling is a fixed constant; it has no parameters to remove.
✗ **C.** The output dimension is set by V (d_v), not by the scaling factor.

Q7 ATTENTION

One query attends over two positions. Its scaled scores are **[2, 0]** and the value vectors are $\mathbf{v_1}$ **= [10]** and $\mathbf{v_2}$ **= [0]**. What is the attention output?

- **A.** 5.0
- **B. ≈ 8.81** ✓
- **C.** 10
- **D.** ≈ 20

Correct answer: B

Why it is correct. Softmax: $w_1 = e^2 / (e^2 + e^0) = 7.389 / 8.389 \approx 0.881$ and $w_2 \approx 0.119$. Output = 0.881·10 + 0.119·0 ≈ **8.81**. Attention is a soft, weighted average of the values.

Why the others fail.

✗ **A.** 5.0 assumes equal weights (0.5 each), ignoring the scores.
✗ **C.** 10 is hard attention (argmax); softmax still gives some weight to the second position.
✗ **D.** ≈ 20 multiplies the raw score by the value without applying softmax.

Q8 ATTENTION

For a sequence of length **n** and model width d, how does the cost of full self-attention scale?

- **A. $O(n^2 \cdot d)$ time and an n × n score matrix in memory** ✓
- **B.** O(n·d) time, linear in sequence length
- **C.** O(log n) time because attention is parallel
- **D.** $O(d^2)$ time, independent of n

Correct answer: A

Why it is correct. Every one of the n queries is compared with all n keys, giving an n × n matrix of scores, each a d-dimensional dot product. This quadratic growth is why long contexts are expensive and why methods like FlashAttention, sparse attention and sliding windows exist.

Why the others fail.

✗ **B.** Linear cost is what RNNs and some efficient-attention variants achieve, not standard attention.
✗ **C.** Parallelism reduces wall-clock time on hardware but not the total amount of work.
✗ **D.** $O(d^2)$ describes the projection matrices, which are only part of the cost.

Q9 MULTI-HEAD

A multi-head attention layer has $\mathbf{d_{model} = 512}$ and **h = 8** heads. What is the per-head dimension d_k, and how many weights (no biases) do the Q, K, V and output projections have in total?

- **A.** 8 and 1,048,576
- **B.** 512 and 8,388,608
- **C. 64 and 1,048,576** ✓
- **D.** 64 and 786,432

Correct answer: C

Why it is correct. $d_k = 512 / 8 = \mathbf{64}$. W_Q, W_K, W_V and W_O are each $512 \times 512 = 262{,}144$, so $4 \times 262{,}144 = \mathbf{1{,}048{,}576}$. Splitting into heads reshapes the projections; it does not add parameters.

Why the others fail.

✗ **A.** 8 is the number of heads, not the dimension of each head.
✗ **B.** Gives every head a full 512-dimensional projection, multiplying the cost by 8.
✗ **D.** 786,432 = 3 × 262,144 forgets the output projection W_O.

Q10 MASKING

What does the **causal (look-ahead) mask** in a decoder's self-attention do?

- **A.** It hides padding tokens from the loss function
- **B. It sets the scores of future positions to −∞ before softmax, so position i can only attend to positions ≤ i** ✓
- **C.** It randomly drops 15% of the input tokens
- **D.** It forces every position to attend only to itself

Correct answer: B

Why it is correct. Adding −∞ to the upper triangle of the score matrix makes those softmax weights exactly 0. During training this lets the model predict every next token in parallel without seeing the answer, which matches how it generates text one token at a time.

Why the others fail.

✗ **A.** That is a padding mask; it has a different purpose.
✗ **C.** Random masking is BERT's pre-training corruption, not the attention mask.
✗ **D.** Only the diagonal and everything before it are allowed; earlier positions are not hidden.

C Transformer Architecture

Q11 POSITIONAL ENCODING

Why does a Transformer need **positional encodings** added to its input embeddings?

- **A.** To normalize the embeddings to unit length
- **B.** To reduce the vocabulary size
- ✓ **C. Self-attention is permutation-equivariant, so without position information the model cannot tell word order**
- **D.** To replace the feed-forward layers

Correct answer: C

Why it is correct. Attention compares tokens as a set: shuffle the input and the outputs are shuffled in the same way. “Dog bites man” and “man bites dog” would look identical. Adding sinusoidal or learned position vectors (or using RoPE) injects order information.

Why the others fail.

✗ **A.** Normalization is done by layer norm, not by positional encodings.
✗ **B.** Positional encodings have nothing to do with tokenization or vocabulary size.
✗ **D.** Feed-forward layers are still present in every block.

Q12 FEED-FORWARD

The position-wise feed-forward network of a Transformer block maps **512 → 2048 → 512** with biases. How many parameters does it have?

- ✓ **A. 2,099,712**
- **B.** 2,097,152
- **C.** 1,050,624
- **D.** 4,194,304

Correct answer: A

Why it is correct. First linear layer: 512·2048 + 2048 = 1,050,624. Second: 2048·512 + 512 = 1,049,088. Total = **2,099,712**. In a typical block the FFN holds about two-thirds of the parameters, twice as many as the attention projections.

Why the others fail.

✗ **B.** 2,097,152 counts the weights only and forgets both bias vectors.
✗ **C.** 1,050,624 is the first linear layer alone.
✗ **D.** 4,194,304 doubles the weight count instead of adding the two layers.

Q13 NORMALIZATION

Why do Transformers use **layer normalization** rather than batch normalization?

- **A.** Layer norm has no learnable parameters
- **B.** Batch norm cannot be used on GPUs
- **C.** Layer norm removes the need for residual connections
- ✓ **D. Layer norm normalizes over each token's features independently, so it does not depend on batch size or on padded, variable-length sequences**

Correct answer: D

Why it is correct. Batch statistics are noisy with small batches and are distorted by padding in variable-length text. Layer norm computes mean and variance across the d_{model} features of each token, so training and inference behave identically.

Why the others fail.

✗ **A.** Layer norm has a learnable scale (γ) and shift (β) per feature.
✗ **B.** Batch norm runs perfectly well on GPUs; it is simply a poor fit for sequences.
✗ **C.** Residual connections and layer norm work together; neither replaces the other.

Q14 CROSS-ATTENTION

In an encoder–decoder Transformer, where do the queries, keys and values of the decoder's **cross-attention** come from?

- **A.** All three come from the encoder output
- ✓ **B. Queries from the decoder; keys and values from the encoder output**
- **C.** Queries from the encoder; keys and values from the decoder
- **D.** All three come from the decoder's previous layer

Correct answer: B

Why it is correct. Each decoder position asks a question (query) about the source sentence, and the encoder states provide what can be matched (keys) and what is retrieved (values). This is how a translation model looks at the source while generating the target.

Why the others fail.

✗ **A.** Then the decoder state would not influence what is attended to.
✗ **C.** This is reversed: the encoder would query the partial output.
✗ **D.** That describes the decoder's masked self-attention, not cross-attention.

Q15 POSITIONAL ENCODING

Using sinusoidal encoding PE(pos, 2i) = sin(pos / $10000^{2i/d}$) and PE(pos, 2i+1) = cos(pos / $10000^{2i/d}$) with $\mathbf{d_{model}} = 4$, what is the vector for **position 0**?

- **A.** [0, 0, 0, 0]
- **B.** [1, 1, 1, 1]
- ✓ **C. [0, 1, 0, 1]**
- **D.** [1, 0, 1, 0]

Correct answer: C

Why it is correct. At pos = 0 every angle is 0. Even dimensions use sin(0) = 0 and odd dimensions use cos(0) = 1, giving **[0, 1, 0, 1]**. Other positions rotate these pairs at different frequencies.

Why the others fail.

✗ **A.** Would be true only if every dimension used sine.
✗ **B.** Would be true only if every dimension used cosine.
✗ **D.** Swaps sine and cosine between even and odd dimensions.

D Pre-trained Language Models

Q16 BERT

What is the main pre-training objective of **BERT**?

- **A.** Predicting the next token from left context only
- **B.** Translating between language pairs
- **C.** Reconstructing the input image from patches
- ✓ **D. Masked language modeling: predicting randomly masked tokens using both left and right context**

Correct answer: D

Why it is correct. BERT is an encoder-only model. Hiding some tokens and predicting them lets every layer attend in both directions, giving deep bidirectional representations. The original paper also used next-sentence prediction, which later work (e.g. RoBERTa) found unnecessary.

Why the others fail.

✗ **A.** Left-to-right next-token prediction is the GPT (causal LM) objective.
✗ **B.** BERT is pre-trained on unlabeled monolingual text, not on translation pairs.
✗ **C.** Masked patch reconstruction is used by vision models such as MAE, not BERT.

Q17 BERT

BERT selects **15%** of tokens for prediction; of those, 80% become [MASK], 10% a random token and 10% stay unchanged. For a **200-token** sequence, about how many tokens are replaced by [MASK]?

- **A.** 30
- **B. 24** ✓
- **C.** 20
- **D.** 3

Correct answer: B

Why it is correct. Selected: 0.15 × 200 = 30. Replaced by [MASK]: 0.8 × 30 = **24**. The 10% random and 10% unchanged tokens reduce the mismatch with fine-tuning, where [MASK] never appears.

Why the others fail.

✗ **A.** 30 is all selected tokens, including the random and unchanged ones.
✗ **C.** 20 is 10% of the sequence; that rate is not used anywhere in the rule.
✗ **D.** 3 is the 10% share of the selected tokens, i.e. the random-token count.

Q18 GPT

You need a model for **open-ended text generation** such as story continuation. Which architecture is the natural fit?

- **A.** An encoder-only model like BERT
- **B.** A static Word2Vec embedding model
- **C. A decoder-only model like GPT trained with causal language modeling** ✓
- **D.** A bidirectional LSTM classifier

Correct answer: C

Why it is correct. A causal decoder is trained to predict the next token given everything before it, exactly the operation repeated during generation. Sampling a token, appending it and predicting again produces text of any length.

Why the others fail.

✗ **A.** BERT sees both directions during training and is not set up to generate left to right.
✗ **B.** Word2Vec produces word vectors; it has no mechanism to generate sequences.
✗ **D.** A classifier outputs a label, and a bidirectional model needs the future text that does not yet exist.

Q19 BERT

When BERT is fine-tuned for **sentence classification**, what is usually fed to the classification head?

- ✓ **A. The final hidden state of the special [CLS] token**
- **B.** The embedding of the last word only
- **C.** The raw token IDs
- **D.** The attention weights of the first layer

Correct answer: A
Why it is correct. [CLS] is prepended to every input. Because it attends to all tokens through every layer, its final vector acts as a summary of the sequence, and a small linear layer on top of it is trained for the task. Mean-pooling all tokens is a common alternative.

Why the others fail.
✗ **B.** The last word's vector mostly represents that word in context, not the whole sentence.
✗ **C.** Token IDs are integers with no learned meaning; the model's outputs are what carry information.
✗ **D.** Attention weights describe where the model looks, not a representation of the content.

Q20 T5

What is the defining idea of the **T5** model?

- **A.** It is an encoder-only model for classification
- **B.** It replaces attention with convolutions
- **C.** It only works for English–German translation
- ✓ **D. Every task is cast as text-to-text: input text in, output text out, with an encoder–decoder model**

Correct answer: D
Why it is correct. T5 uses task prefixes such as "summarize:" or "translate English to German:", and even classification answers are generated as words ("positive"). One model, one loss and one decoding procedure serve all tasks.

Why the others fail.
✗ **A.** T5 has both an encoder and a decoder.
✗ **B.** T5 is a standard Transformer built on attention.
✗ **C.** Translation is only one of many tasks T5 handles in the same format.

E Fine-tuning & Decoding

Q21 LORA

LoRA adapts a frozen weight matrix W of size **4096 × 4096** by learning W + BA, with B (4096 × r) and A (r × 4096) and rank **r = 8**. How many trainable parameters does this add?

- **A.** 16,777,216
- **B. 65,536** ✓
- **C.** 32,768
- **D.** 131,072

Correct answer: B
Why it is correct. Trainable = r(d + k) = 8 × (4096 + 4096) = **65,536**, about 0.4% of the 16.8 million in W. The base weights stay frozen, so many task adapters can share one model.

Why the others fail.
✗ **A.** 16,777,216 = 4096² is the full matrix, i.e. ordinary fine-tuning.
✗ **C.** 32,768 counts only one of the two low-rank matrices.
✗ **D.** 131,072 doubles the correct value.

Q22 TRAINING

During training of a sequence-to-sequence decoder, **teacher forcing** means:

- **A.** A larger teacher model labels the data
- **B.** The learning rate is reduced when validation loss rises
- **C. The decoder receives the ground-truth previous token as input instead of its own previous prediction** ✓
- **D.** Gradients are clipped to a fixed norm

Correct answer: C
Why it is correct. Feeding the correct prefix stabilizes and parallelizes training. The drawback is exposure bias: at inference time the model must consume its own predictions and has never practised recovering from its own mistakes.

Why the others fail.
✗ **A.** That is knowledge distillation (or pseudo-labelling), a different technique.
✗ **B.** That is a learning-rate schedule such as reduce-on-plateau.
✗ **D.** That is gradient clipping, used against exploding gradients.

Q23 DECODING

How does **beam search** with beam width k differ from greedy decoding?

- ✓ **A. It keeps the k most probable partial sequences (by cumulative log-probability) at every step instead of only the single best token**
- **B.** It samples tokens randomly in proportion to their probability
- **C.** It always finds the globally most probable sequence
- **D.** It processes k sentences in parallel batches

Correct answer: A

Why it is correct. Greedy decoding commits to the best token at each step and can miss a sequence whose first token looked slightly worse. Beam search explores k hypotheses, usually with a length penalty, which helps translation and summarization quality.

Why the others fail.

✗ **B.** That is ancestral sampling; beam search is deterministic.
✗ **C.** Beam search is still a heuristic; exact search over all sequences is intractable.
✗ **D.** Batching is an implementation detail; beams are hypotheses for the same input.

Q24 DECODING

A model's logits for two tokens are **[2, 1]**. Using sampling temperature **T = 0.5** (logits divided by T), what is the probability of the first token?

- **A.** ≈ 0.731
- **B.** ≈ 0.622
- **C.** 1.0
- ✓ **D. ≈ 0.881**

Correct answer: D

Why it is correct. Divide by T: [4, 2]. Softmax: $p_1 = e^4 / (e^4 + e^2) = 1 / (1 + e^{-2}) \approx$ **0.881**. Low temperature sharpens the distribution; high temperature flattens it.

Why the others fail.

✗ **A.** ≈ 0.731 is the probability at T = 1 (no scaling).
✗ **B.** ≈ 0.622 is the probability at T = 2, which flattens the distribution.
✗ **C.** Probability 1.0 is reached only in the limit T → 0 (greedy).

Q25 DECODING

With **nucleus (top-p) sampling, p = 0.8**, and sorted next-token probabilities **[0.50, 0.25, 0.15, 0.10]**, how many tokens are kept in the candidate set?

- **A.** 1
- **B.** 2
- **C. 3** ✓
- **D.** 4

Correct answer: C

Why it is correct. Top-p keeps the smallest set of most-likely tokens whose cumulative probability reaches p. 0.50 + 0.25 = 0.75 < 0.8, so add the third: 0.90 ≥ 0.8. **3** tokens are kept and renormalized before sampling.

Why the others fail.

✗ **A.** One token is greedy decoding; 0.50 is well below 0.8.
✗ **B.** Two tokens reach only 0.75, short of the 0.8 threshold.
✗ **D.** The fourth token is not needed because 0.8 is already reached.

F Evaluation & Efficiency

Q26 PERPLEXITY

A language model assigns probability **0.25** to every correct next token in a test set. What is its **perplexity**?

- **A. 4** ✓
- **B.** 0.25
- **C.** ≈ 1.386
- **D.** 16

Correct answer: A

Why it is correct. Perplexity = exp(average negative log-likelihood) = exp(−ln 0.25) = 1 / 0.25 = **4**. The model is as uncertain as if it were choosing uniformly among 4 tokens at each step. Lower is better.

Why the others fail.

✗ **B.** 0.25 is the per-token probability, not the perplexity.
✗ **C.** ≈ 1.386 = ln 4 is the cross-entropy in nats, before exponentiating.
✗ **D.** 16 squares the correct value.

Q27 BLEU

What does the **BLEU** score used for machine translation measure?

- **A.** Semantic similarity using contextual embeddings
- **B.** The fraction of reference words that appear in the output (recall only)
- **C.** The perplexity of the translation
- ✓ **D. Modified n-gram precision of the output against reference translations, with a brevity penalty for short outputs**

Correct answer: D

Why it is correct. BLEU combines clipped precision for 1- to 4-grams (geometric mean) and multiplies by a brevity penalty so that very short outputs cannot score high. It rewards surface overlap and can miss valid paraphrases.

Why the others fail.

✗ **A.** That describes embedding-based metrics such as BERTScore.
✗ **B.** BLEU is precision-oriented; recall-oriented overlap is the idea behind ROUGE.
✗ **C.** Perplexity evaluates a language model's probabilities, not a translation against a reference.

Q28 METRICS

A named-entity recognizer produces **8 true positives, 2 false positives and 4 false negatives**. What is its F1 score?

- **A.** 0.800
- **B.** 0.667
- **C.** 0.733
- ✓ **D. ≈ 0.727**

Correct answer: D

Why it is correct. Precision = 8 / (8 + 2) = 0.800. Recall = 8 / (8 + 4) = 0.667. F1 = 2PR / (P + R) = 2·0.8·0.667 / 1.467 ≈ **0.727**. The harmonic mean sits closer to the weaker of the two.

Why the others fail.

✗ **A.** 0.800 is the precision.
✗ **B.** 0.667 is the recall.
✗ **C.** 0.733 is the arithmetic mean of P and R, not the harmonic mean.

Q29 INFERENCE

What does the **KV cache** do during autoregressive inference with a decoder?

- **A.** It caches the final output text to answer repeated prompts
- **B.** It compresses model weights to 8-bit integers
- **C. It stores the keys and values of previous tokens, so each new step only computes K and V for the newest token**
- **D.** It removes the need for positional encodings

Correct answer: C

Why it is correct. Without the cache, generating token t would recompute attention projections for all $t - 1$ earlier tokens. Caching them reduces the per-step work, at the cost of memory that grows with layers × heads × sequence length.

Why the others fail.

✗ **A.** That is response caching at the application level.
✗ **B.** That is quantization, a separate optimization.
✗ **D.** Positions still need to be encoded; the cache only reuses computed keys and values.

Q30 EFFICIENCY

A model's context length is doubled from **2,048 to 4,096** tokens. By what factor does the memory for one full attention score matrix grow?

- **A.** 2
- **B. 4**
- **C.** 8
- **D.** 16

Correct answer: B

Why it is correct. The score matrix is n × n. Doubling n gives $(2n)^2 = 4n^2$, so memory grows by a factor of **4**. This quadratic scaling is the main obstacle to long contexts with standard attention.

Why the others fail.

✗ **A.** Linear growth applies to per-token quantities such as activations of the FFN, not the n × n matrix.
✗ **C.** 8 would correspond to cubic scaling.
✗ **D.** 16 would be the factor if the length were quadrupled.

✓ Answer Key & Score Sheet

Q	Ans	Topic	Q	Ans	Topic
1	**B**	Tokenization & Embeddings	16	**D**	Pre-trained Models
2	**C**	Tokenization & Embeddings	17	**B**	Pre-trained Models
3	**D**	Tokenization & Embeddings	18	**C**	Pre-trained Models
4	**A**	Tokenization & Embeddings	19	**A**	Pre-trained Models
5	**A**	Tokenization & Embeddings	20	**D**	Pre-trained Models
6	**D**	Attention	21	**B**	Fine-tuning & Decoding
7	**B**	Attention	22	**C**	Fine-tuning & Decoding
8	**A**	Attention	23	**A**	Fine-tuning & Decoding
9	**C**	Attention	24	**D**	Fine-tuning & Decoding
10	**B**	Attention	25	**C**	Fine-tuning & Decoding
11	**C**	Transformer	26	**A**	Evaluation & Efficiency
12	**A**	Transformer	27	**D**	Evaluation & Efficiency
13	**D**	Transformer	28	**D**	Evaluation & Efficiency
14	**B**	Transformer	29	**C**	Evaluation & Efficiency
15	**C**	Transformer	30	**B**	Evaluation & Efficiency

Score interpretation

Score	Level	Recommended action
26–30	Mastery	Move on to Module 04; try explaining each answer aloud.
20–25	Proficient	Review the sections behind your misses, then redo those questions.
13–19	Developing	Re-read the Theory Study Guide sections, work the numeric examples by hand, retake after 48 hours.
0–12	Foundation	Restart with Sections 1 and 2 of the guide, then attempt the quiz again.

KEY IDEA
Track your misses by topic. If you missed 2+ questions in one part (A–F), that part is your priority for revision. Numeric questions (Q2, Q4, Q7, Q9, Q12, Q17, Q21, Q24, Q25, Q26, Q28, Q30) can all be solved with the formula sheet in Section 7 of the Module 03 Study Guide.